Министерство науки и высшего образования Российской Федерации

Федеральное государственное бюджетное образовательное учреждение высшего образования

«Российский экономический университет им. Г.В. Плеханова»

Практическая работа №2

Поддержка и тестирование программных модулей
на тему: Создание приложения калькулятор

Специальность «Информационные системы и программирование»

Руководитель

_____ / Д.Б. Берестов/

Оценка _____

«_02_» _октября_ 2026 г.

Исполнитель

_____ / М.Т. Мурасов/

Группа ИПо-32

«_02_» _октября_ 2026 г.

Пермь, 2026

Оглавление

Введение 2
В данной практической работе я использовал 2
1. Создание решения 3
2. Создание библиотеки классов 4
3. Логика калькулятора 5
4. Создание WPF-приложения 7
5. Создание интерфейса калькулятора 8
6. Написание логики интерфейса 9
7. Создание проекта модульных тестов 12
8. Запуск тестов 15
Заключение 16

Введение

Visual Studio - интегрированная среда разработки для создания приложений. В практической работе разработано настольное приложение-калькулятор на языке C# с графическим интерфейсом WPF, отдельной библиотекой вычислений и модульными тестами.

В данной практической работе я использовал:

1. Среда разработки: Microsoft Visual Studio 2022/2026 и .NET SDK.

2. Технологии: WPF для интерфейса, .NET 8 для выполнения приложения, MSTest для автоматической проверки.

3. Язык: C# - объектно-ориентированный язык, на котором написаны вычисления и обработчики интерфейса.

4. Архитектура: решение разделено на библиотеку MurasovLibraryFramework, приложение MurasovWPF и тесты MurasovTests.

Ход работы

1. Создание решения

Сначала было создано решение MurasovCalc. Оно объединяет три проекта и позволяет независимо развивать вычислительную библиотеку, пользовательский интерфейс и набор модульных тестов.

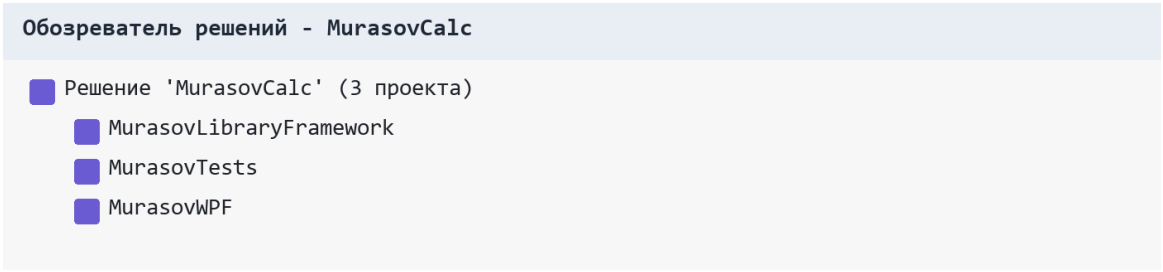


Рисунок 1 - Структура решения MurasovCalc

Такое разделение упрощает сопровождение программы: интерфейс не содержит базовую арифметическую логику, а правильность библиотеки можно проверять без запуска окна.

2. Создание библиотеки классов

В решение добавлен проект MurasovLibraryFramework. В нём создан класс MurasovClass, содержащий методы выполнения арифметических операций.

Проект библиотеки классов

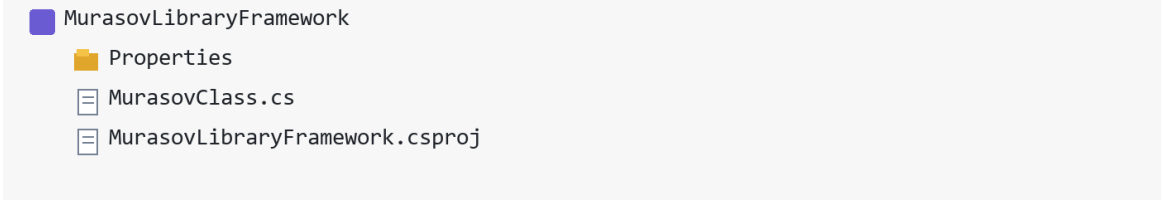


Рисунок 2 - Состав проекта библиотеки

MurasovLibraryFramework.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <RootNamespace>MurasovLibraryFramework</RootNamespace>
    <AssemblyName>MurasovLibraryFramework</AssemblyName>
    <GenerateAssemblyInfo>false</GenerateAssemblyInfo>
  </PropertyGroup>
</Project>

```

Рисунок 3 - Параметры проекта библиотеки

3. Логика калькулятора

Статический метод Execute принимает два числа и символ операции. Конструкция switch выполняет сложение, вычитание, умножение, деление или возведение в степень. При делении на ноль возвращается положительная бесконечность.

MurasovClass.cs - метод Execute

```
using System;

namespace MurasovLibraryFramework
{
    public class MurasovClass
    {
        //метод ариф операций
        public static dynamic Execute(dynamic first, char op, dynamic second)
        {
            dynamic result = 0;
            switch (op)
            {
                case '+':
                    result = first + second; break;
                case '-':
                    result = first - second; break;
                case '*':
                    result = first * second; break;
                case '/':
                    //проверка деления на ноль
                    if (second == 0)
                    {
                        result = double.PositiveInfinity; //возвращаем бесконечность
                    }
                    else
                    {
                        result = first / second;
                    }
                    break;
                case '^':
                    result = Math.Pow(first, second); break;
                default:
                    result = 0; break;
            }
```

Рисунок 4 - Реализация метода Execute

Метод Calculate

Для удобного вызова из WPF-приложения добавлен экземплярный метод Calculate. Он преобразует строковое обозначение операции в символ, вызывает Execute и возвращает результат типа double.

MurasovClass.cs - метод Calculate

```
                    break;
                case '^':
                    result = Math.Pow(first, second); break;
                default:
                    result = 0; break;
            }
            return result;
        }
        //метод для вызова из WPF
        public double Calculate(double num1, double num2, string operation)
        {
            char op = operation[0];
            dynamic result = Execute(num1, op, num2);
            return Convert.ToDouble(result);
        }
    }
}
```

Рисунок 5 - Метод Calculate

Библиотека не зависит от интерфейса, поэтому может повторно использоваться в других приложениях и тестироваться отдельно.

4. Создание WPF-приложения

Для графического интерфейса создан проект MurasovWPF. В него входят точка входа App.xaml, окно MainWindow и программная логика окна.

Проект графического интерфейса

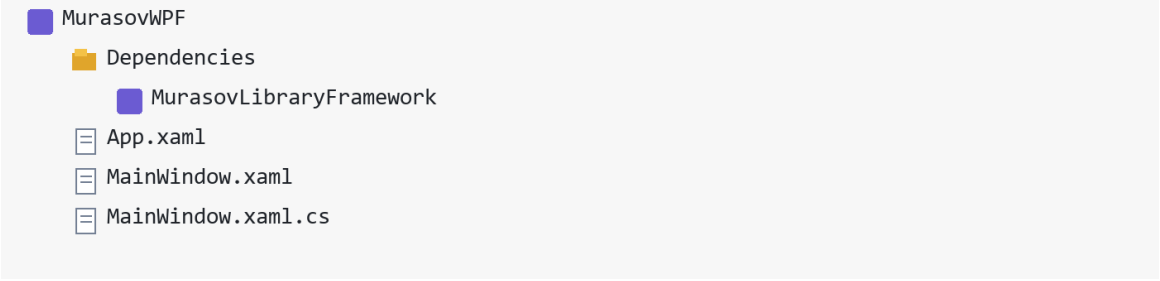


Рисунок 6 - Состав WPF-проекта

В проект добавлена ссылка на MurasovLibraryFramework, поэтому методы библиотеки доступны обработчикам пользовательского интерфейса.

MurasovWPF.csproj - ссылка на библиотеку

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>WinExe</OutputType>
    <TargetFramework>net8.0-windows</TargetFramework>
    <UseWPF>true</UseWPF>
    <RootNamespace>MurasovWPF</RootNamespace>
    <AssemblyName>MurasovWPF</AssemblyName>
    <GenerateAssemblyInfo>false</GenerateAssemblyInfo>
  </PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="..\MurasovLibraryFramework\MurasovLibraryFramework.csproj" />
  </ItemGroup>
</Project>

```

Рисунок 7 - Ссылка WPF-проекта на библиотеку

5. Создание интерфейса калькулятора

Главное окно содержит два поля вывода и сетку кнопок. Пользователь может вводить цифры, десятичную точку, скобки и операции +, -, *, /, ^. Предусмотрены очистка, удаление последнего символа и вычисление результата.

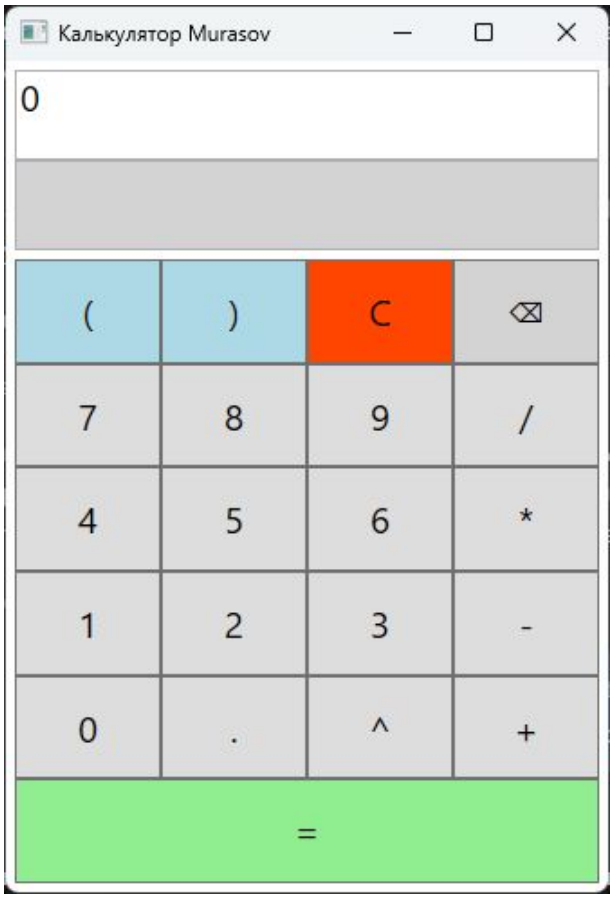


Рисунок 8 - Интерфейс калькулятора

6. Написание логики интерфейса

В файле MainWindow.xaml.cs хранится текущее выражение и реализованы обработчики кнопок. При нажатии цифры или операции строка выражения обновляется и отображается пользователю.

MainWindow.xaml.cs - обработчики

```
using System;
using System.Windows;
using System.Windows.Controls;
using MurasovLibraryFramework;  //моя библиотека

namespace MurasovWPF
{
    public partial class MainWindow : Window
    {
        //хранение выражения
        string expr = "";

        MurasovClass calc = new MurasovClass();

        //конструктор окна
        public MainWindow()
        {
            InitializeComponent();
        }

        //обработчик цифр
        private void Button_Click(object sender, RoutedEventArgs e)
        {
            Button btn = (Button)sender;

            expr = expr + btn.Content.ToString();
            InputTextBox.Text = expr;
        }

        //обработчик точки
        private void Dot_Click(object sender, RoutedEventArgs e)
        {
            expr = expr + ".";
            InputTextBox.Text = expr;
        }

        //обработчик скобок
        private void LeftParenthesis_Click(object sender, RoutedEventArgs e)
```

Рисунок 9 - Обработчики событий MainWindow.xaml.cs

Разметка окна

Интерфейс описан декларативно в XAML. Сетка состоит из шести строк и четырёх столбцов, а обработчики Click связывают кнопки с программной логикой.

MainWindow.xaml - разметка

```
<Window x:Class="MurasovWPF.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Калькулятор Murasov" Height="500" Width="350"
        WindowStartupLocation="CenterScreen">

    <Grid Margin="5">
        <Grid.RowDefinitions>
            <RowDefinition Height="50"/>
            <RowDefinition Height="50"/>
            <RowDefinition Height="*"/>
        </Grid.RowDefinitions>

        <!-- поле ввода чисел и результата -->
        <TextBox x:Name="InputTextBox"
                 Grid.Row="0"
                 FontSize="20"
                 IsReadOnly="True"
                 Background="White"
                 Text="0"/>

        <!-- поле отображения вычисления -->
        <TextBox x:Name="ResultTextBox"
                 Grid.Row="1"
                 FontSize="14"
                 IsReadOnly="True"
                 Background="LightGray"/>

        <!-- сетка с кнопками -->
        <Grid Grid.Row="2" Margin="0,5,0,0">
            <Grid.RowDefinitions>
                <RowDefinition Height="*"/>
```

Рисунок 10 - Разметка MainWindow.xaml

Запуск и проверка калькулятора

После сборки решение было запущено. Для проверки выбрана операция возведения в степень: выражение 2 ^ 3 должно вернуть 8.

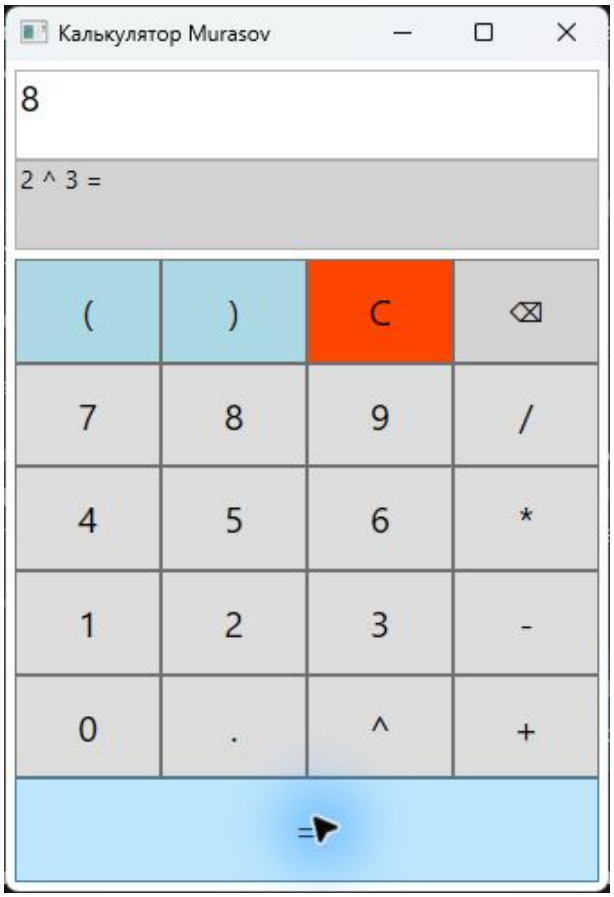


Рисунок 11 - Результат вычисления 2 ^ 3 = 8

Результат получен корректно. Окно также поддерживает выражения со скобками, десятичные числа и приоритет арифметических операций.

7. Создание проекта модульных тестов

Для автоматической проверки создан проект MurasovTests. Он использует фреймворк MSTest и содержит ссылку на тестируемую библиотеку.

MurasovTests.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <RootNamespace>MurasovTests</RootNamespace>
    <AssemblyName>MurasovTests</AssemblyName>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
    <GenerateAssemblyInfo>false</GenerateAssemblyInfo>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.14.1" />
    <PackageReference Include="MSTest.TestAdapter" Version="3.10.4" />
    <PackageReference Include="MSTest.TestFramework" Version="3.10.4" />
  </ItemGroup>
  <ItemGroup>
    <ProjectReference Include="..\MurasovLibraryFramework\MurasovLibraryFramework.csproj" />
  </ItemGroup>
</Project>

```

Рисунок 12 - Конфигурация проекта модульных тестов

Параметр IsTestProject сообщает среде сборки, что проект содержит тесты. Пакеты TestAdapter и TestFramework обеспечивают обнаружение и выполнение методов с атрибутом TestMethod.

Зависимости тестов

Проект тестов ссылается на библиотеку вычислений и пакеты MSTest. Благодаря этому тесты вызывают MurasovClass.Execute и сравнивают фактические результаты с ожидаемыми.

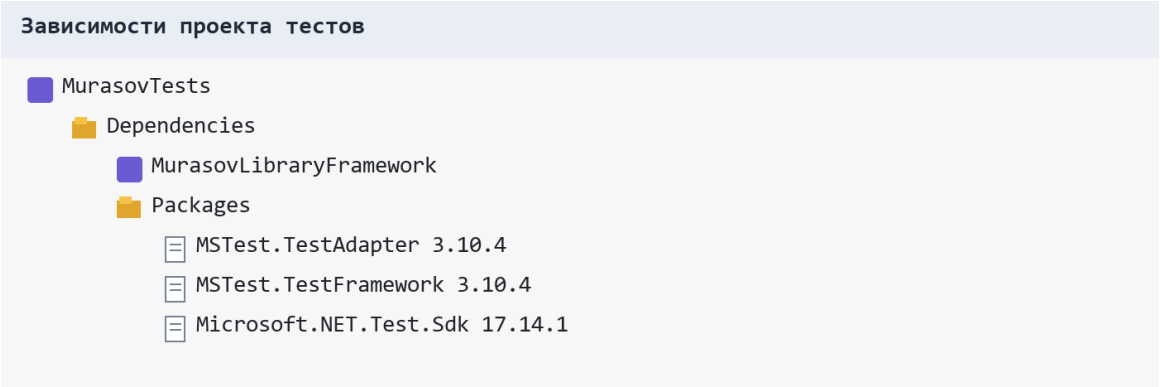


Рисунок 13 - Зависимости MurasovTests

Всего подготовлено семь тестовых сценариев: пять основных операций, деление на ноль и вызов метода Calculate.

Код модульных тестов

Каждый тест отмечен атрибутом [TestMethod], а класс - атрибутом [TestClass]. Проверка выполняется методом Assert.AreEqual.

MurasovClassTests.cs

```
using Microsoft.VisualStudio.TestTools.UnitTesting;
using MurasovLibraryFramework;// моя библиотека

namespace MurasovTests
{
    [TestClass]
    public class MurasovClassTests
    {
        //сложение
        [TestMethod]
        public void test_3plus3()
        {
            dynamic result = MurasovClass.Execute(3, '+', 3);
            Assert.AreEqual(6, result);
        }

        //вычитание
        [TestMethod]
        public void test_10minus4()
        {
            dynamic result = MurasovClass.Execute(10, '-', 4);
            Assert.AreEqual(6, result);
        }

        //умножение
        [TestMethod]
        public void test_6times7()
        {
            dynamic result = MurasovClass.Execute(6, '*', 7);
            Assert.AreEqual(42, result);
        }

        //деление
        [TestMethod]
        public void test_15divide3()
        {
            dynamic result = MurasovClass.Execute(15, '/', 3);
            Assert.AreEqual(5, result);
        }

        //степень
        [TestMethod]
        public void test_2to3()
        {
            dynamic result = MurasovClass.Execute(2, '^', 3);
```

Рисунок 14 - Класс MurasovClassTests

8. Запуск тестов

Тесты запущены командой dotnet test в конфигурации Release. Среда обнаружила один тестовый модуль и выполнила все семь проверок.

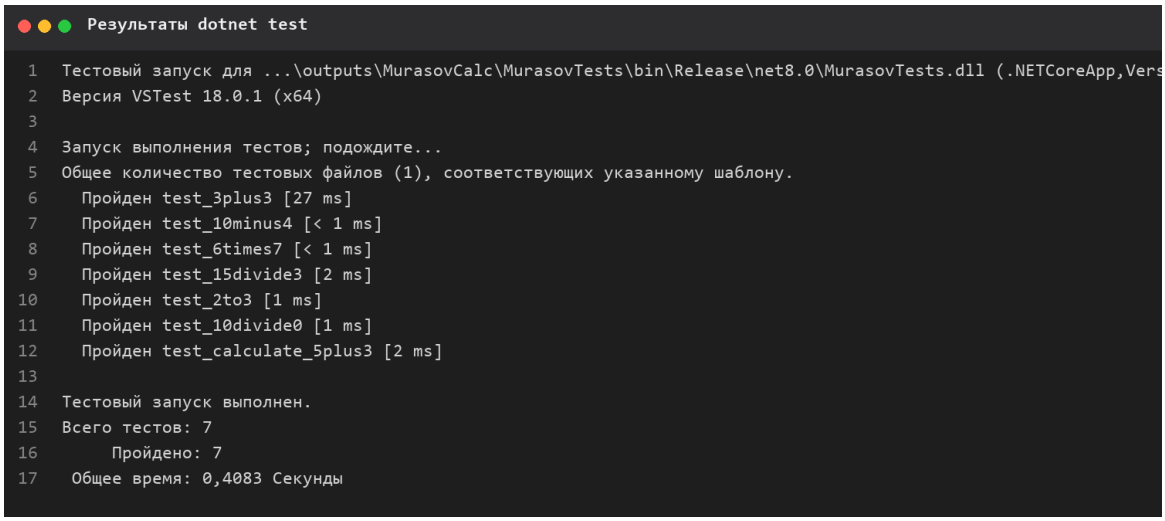


Рисунок 15 - Результаты тестирования

Итог: не пройдено - 0, пройдено - 7, пропущено - 0. Сборка завершена без ошибок. При проверке индекса уязвимостей NuGet получены предупреждения NU1900 из-за недоступности сервиса. Все тесты библиотеки MurasovLibraryFramework успешно выполнены.

Заключение

В ходе практической работы было разработано приложение «Калькулятор» с графическим интерфейсом WPF. Решение разделено на библиотеку вычислений, приложение и проект модульных тестов.

В процессе работы были закреплены навыки создания многопроектного решения, разработки интерфейса на XAML, обработки событий, реализации арифметических выражений, подключения проектных зависимостей и автоматического тестирования.

Приложение поддерживает сложение, вычитание, умножение, деление, возведение в степень и скобки. Для защиты от некорректного результата реализована обработка деления на ноль.

Все семь модульных тестов успешно пройдены. Проект MurasovCalc собирается без ошибок. Поставленная задача выполнена полностью.